

Dokumentacja projektu

Calisia Banking App

Aplikacja bankowa z panelem klienta, produktami finansowymi
i modelem CQRS

Data raportu: 06.06.2026

Autorzy:

Kubryn Jeremiasz

Łaszuk Jakub

Makarevich Ksenia

Nuckowski Michał

Ruczyńska Nikola

Wołczko Jakub

Olszewski Dawid

Spis treści

1 Cel i zakres aplikacji

Calisia Banking App jest aplikacją bankową przeznaczoną do obsługi podstawowych czynności klienta banku. System umożliwia rejestrację i logowanie klienta, prezentację dashboardu finansowego, zarządzanie produktami bankowymi, wykonywanie przelewów, obsługę kart oraz kredytu gotówkowego.

Aplikacja składa się z dwóch głównych części:

- frontendu typu SPA zbudowanego w React i TypeScript,
- backendu REST API zbudowanego w .NET, podzielonego na warstwy API, Application, Domain, Infrastructure oraz ReadModel.

System korzysta z dwóch baz danych SQL Server. Baza zapisu `CalisiaWriteDb` przechowuje model domenowy oraz outbox, natomiast baza odczytu `CalisiaReadDb` przechowuje zoptymalizowane dane dashboardu.

2 Najważniejsze funkcje systemu

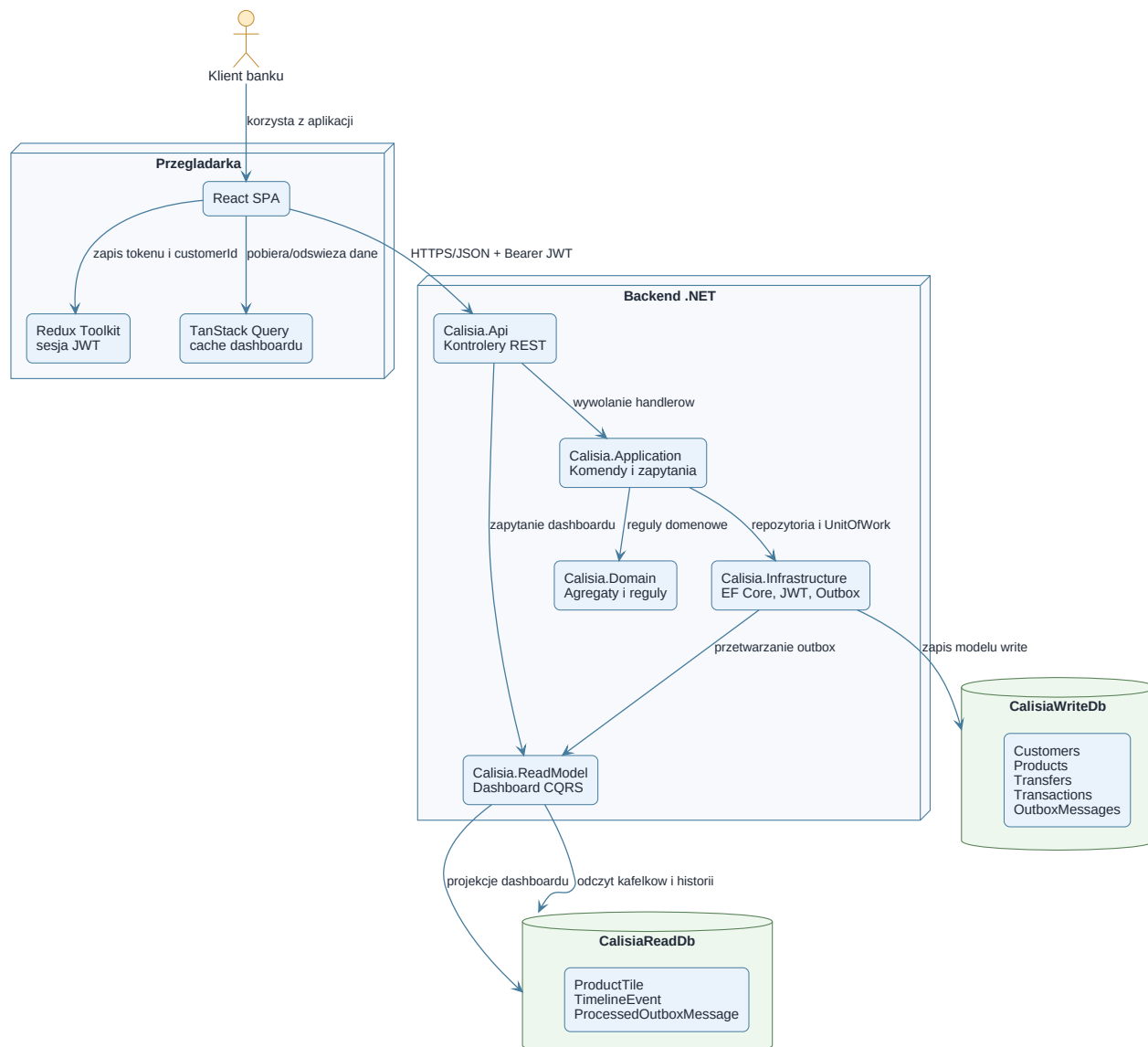
Obszar	Opis funkcjonalności
Rejestracja klienta	Utworzenie konta klienta, zahashowanie hasła oraz automatyczne dodanie produktów startowych: konta głównego i karty debetowej. Dla klienta prestiżowego tworzona jest także karta kredytowa.
Logowanie	Weryfikacja danych dostępowych i wydanie tokenu JWT używanego przez frontend do autoryzowanych żądań.
Dashboard	Prezentacja salda, listy produktów i historii zdarzeń finansowych. Dane pochodzą z modelu odczytu.
Produkty bankowe	Obsługa kont, kart oraz kredytów. Produkty są przechowywane w jednej tabeli <code>Products</code> z rozróżnieniem typu przez kolumnę <code>Discriminator</code> .
Przelewy	Obsługa przelewów własnych oraz zewnętrznych. Przelew własny aktualizuje saldo źródła i celu, a przelew zewnętrzny tylko obciąża konto źródłowe.
Kredyt gotówkowy	Dodanie produktu typu kredyt gotówkowy zwiększa saldo konta głównego. Wcześniejsza spłata pobiera kwotę kredytu z konta głównego i usuwa produkt kredytu.
Karty	Symulacja płatności kartą debetową pobiera środki z konta głównego. Symulacja płatności kartą kredytową zmniejsza dostępne saldo karty. Spłata karty kredytowej pobiera z konta głównego wykorzystany limit.

3 Architektura

Backend stosuje podział zbliżony do Clean Architecture. Warstwa API odpowiada za kontrolery HTTP i autoryzację, Application za przypadki użycia, Domain za reguły biznesowe,

Infrastructure za dostęp do bazy, JWT i outbox, a ReadModel za projekcje dashboardu.

Calisia - architektura systemu



Rysunek 1: Architektura systemu Calisia

Diagram przedstawia główne moduły systemu, przepływ żądań z aplikacji React do API oraz rozdzielenie bazy zapisu i bazy odczytu.

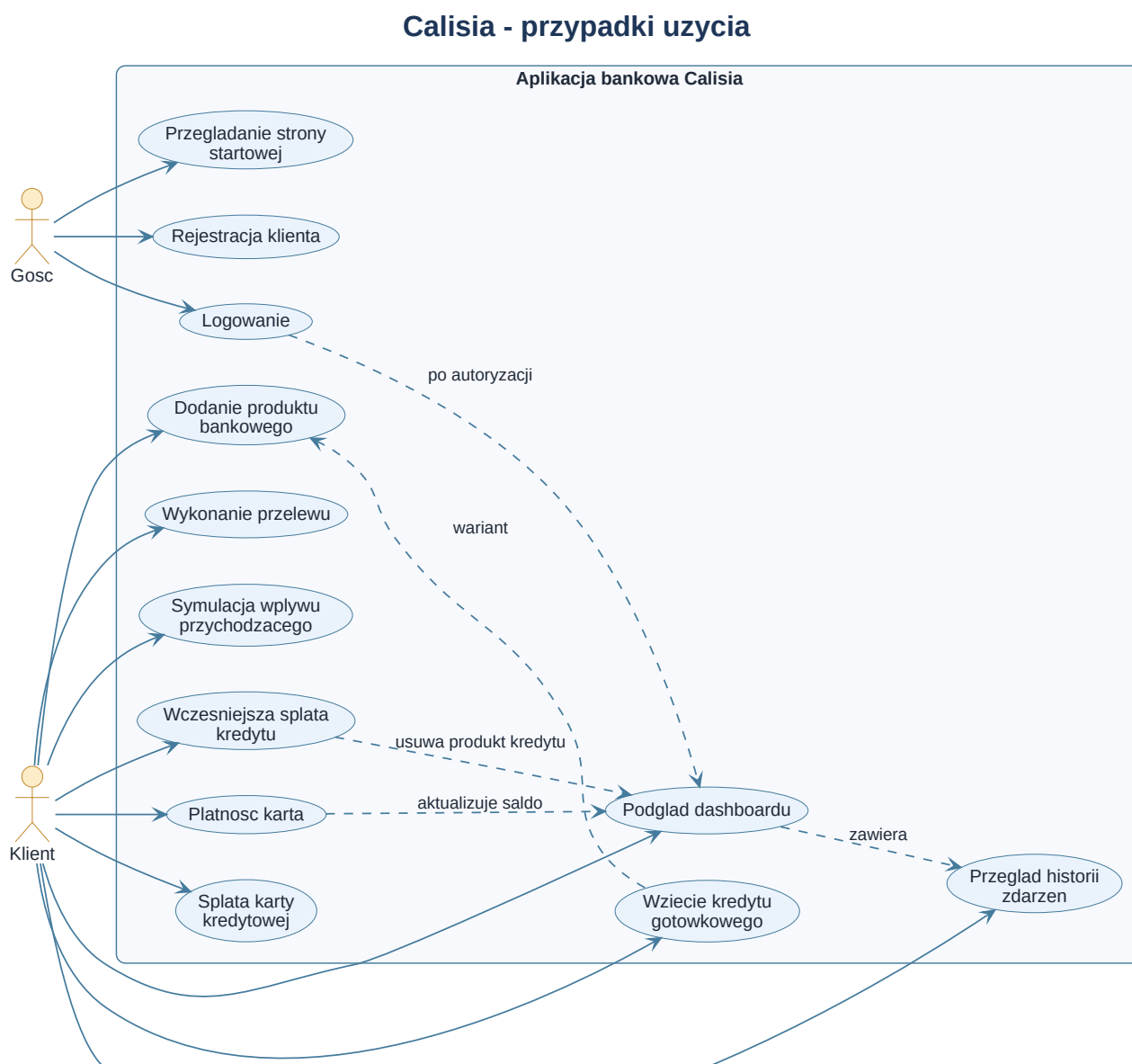
3.1 Warstwy backendu

Warstwa	Odpowiedzialność
Calisia.Api	Kontrolery REST, middleware obsługi wyjątków, konfiguracja CORS, JWT i Swagger/OpenAPI.
Calisia.Contracts	Kontrakty żądań i odpowiedzi HTTP, między innymi rejestracja, logowanie, produkty, karty i przelewy.
Calisia.Application	Komendy i zapytania aplikacyjne, walidacja przypadków użycia oraz koordynacja repozytoriów i jednostki pracy.
Calisia.Domain	Agregaty, encje, value objects, enumy i zdarzenia domenowe.

Warstwa	Odpowiedzialność
Calisia.Infrastructure	Implementacje repozytoriów EF Core, konfiguracje mapowania, hashowanie haseł, tokeny JWT oraz procesor outbox.
Calisia.ReadModel	Odczyt dashboardu, modele ProductTile i TimelineEvent oraz projekcje zdarzeń domenowych.

4 Przypadki użycia

Diagram przypadków użycia pokazuje czynności dostępne dla gościa oraz zalogowanego klienta.



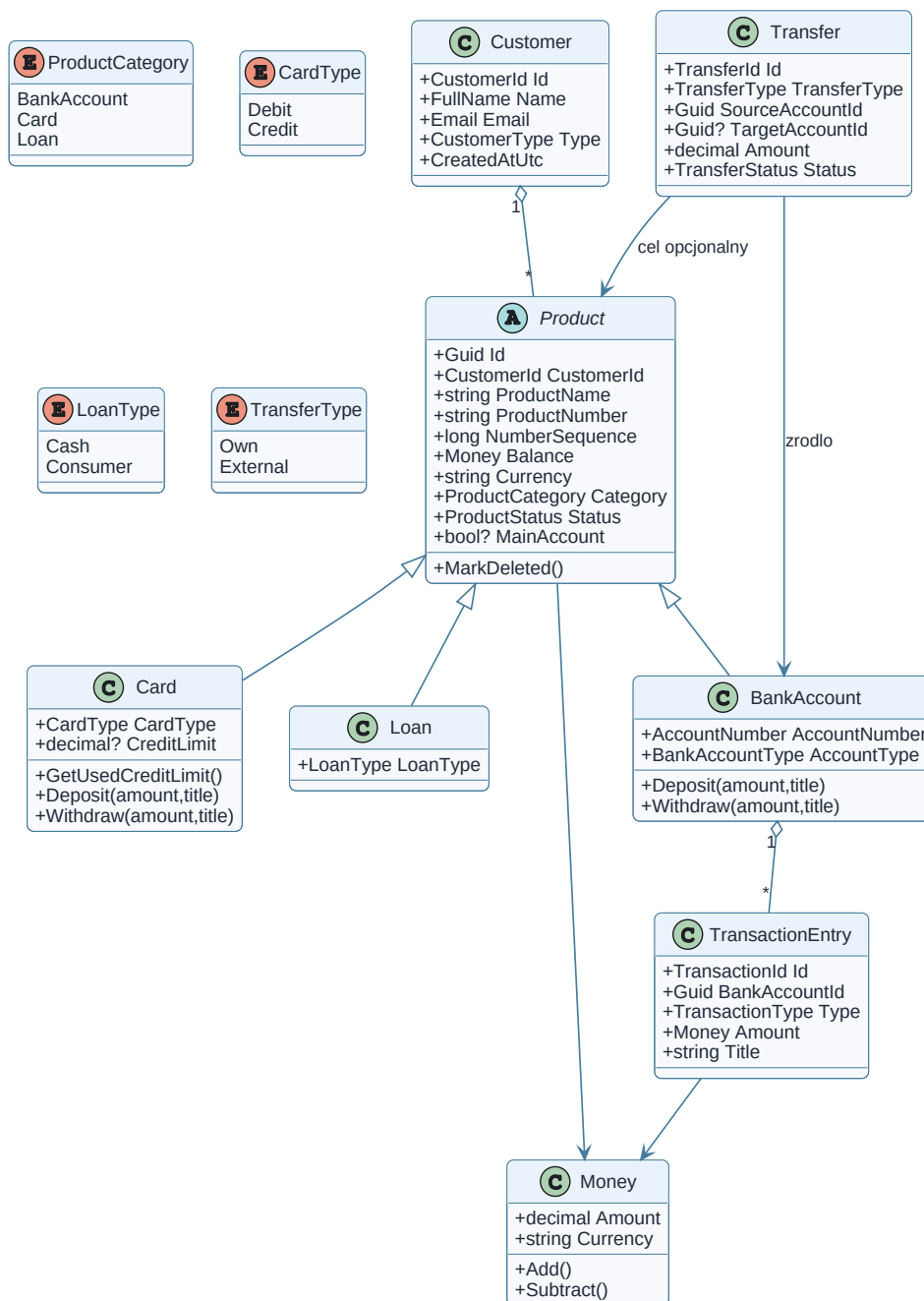
Rysunek 2: Przypadki użycia aplikacji

Diagram pokazuje funkcje dostępne dla gościa oraz operacje wykonywane po zalogowaniu klienta banku.

5 Model domenowy

Centralnym elementem domeny jest abstrakcyjny produkt bankowy **Product**. Dziedziczą po nim: **BankAccount**, **Card** i **Loan**. Takie podejście pozwala przechowywać różne produkty finansowe w jednej tabeli, przy zachowaniu osobnych reguł biznesowych dla kont, kart i kredytów.

Calisia - model domenowy



Rysunek 3: Model domenowy

Diagram prezentuje najważniejsze agregaty domenowe: klienta, produkty bankowe, przelewy, transakcje oraz obiekt wartości pieniędzy.

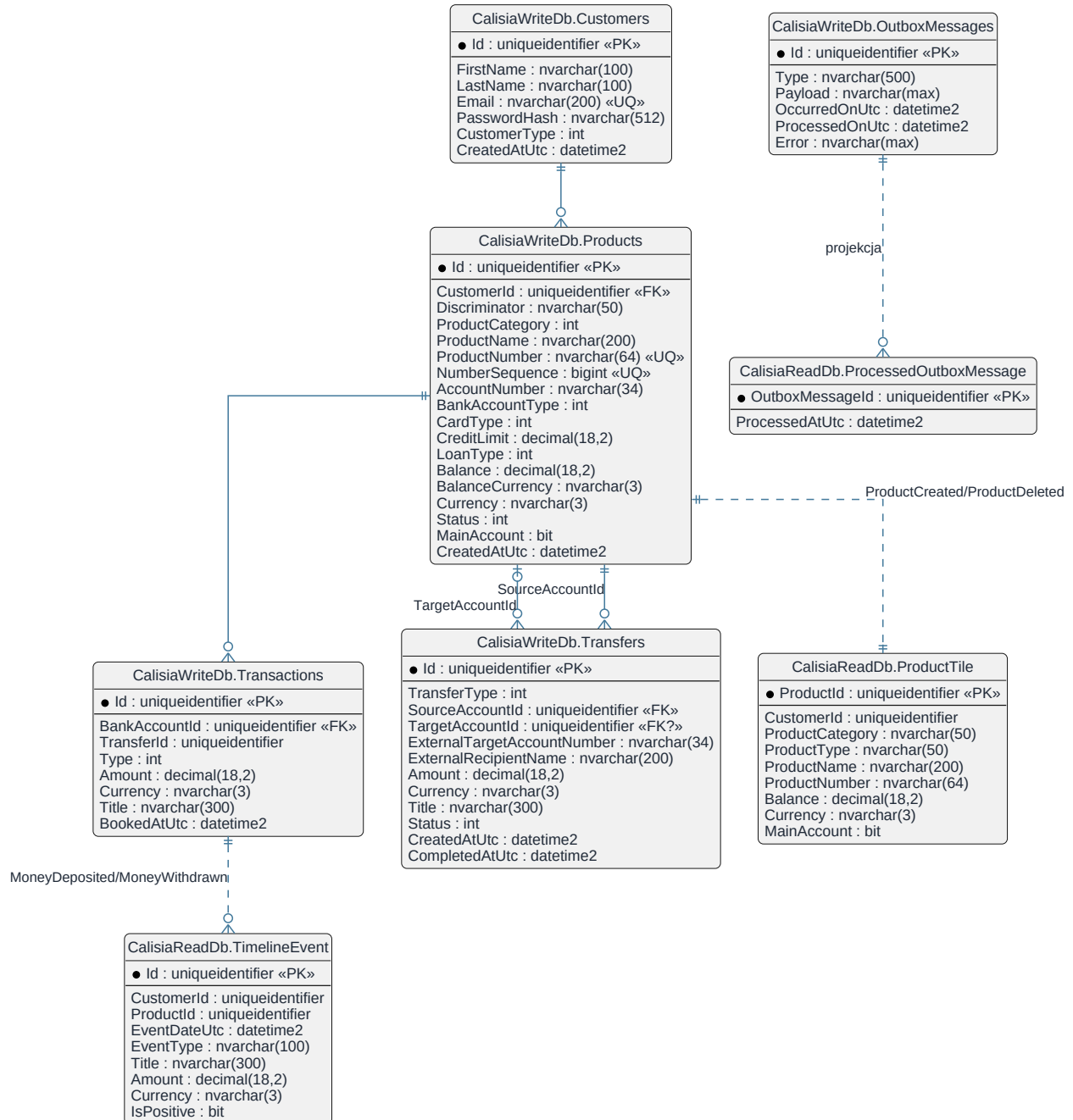
5.1 Wybrane reguły domenowe

- Konto bankowe może przyjmować wpłaty i wypłaty, a każda operacja generuje wpis transakcji oraz zdarzenie domenowe.
- Karta debetowa nie posiada limitu kredytowego i przy płatności obciąża konto główne klienta.
- Karta kredytowa posiada `CreditLimit`; jej saldo reprezentuje dostępny limit. Płatność zmniejsza saldo karty, a spłata zwiększa je do wysokości limitu.
- Kredyt gotówkowy jest produktem typu `Loan`. Wzięcie kredytu zasila konto główne, natomiast wcześniejsza spłata pobiera kwotę kredytu z konta głównego i usuwa produkt kredytu.
- Przelew własny aktualizuje dwa produkty: źródłowy i docelowy. Przelew zewnętrzny aktualizuje tylko produkt źródłowy.

6 Schemat bazy danych

System korzysta z dwóch baz: zapisu i odczytu. Baza zapisu przechowuje pełny stan domenowy, a baza odczytu przechowuje uproszczony widok potrzebny dashboardowi.

Calisia - schemat baz danych



Rysunek 4: Schemat baz danych write/read

Diagram opisuje relacje między tabelami modelu zapisu oraz tabelami projekcji dashboardu w modelu odczytu.

6.1 CalisiaWriteDb

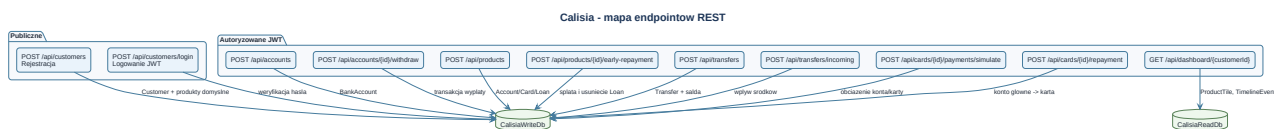
Tabela	Znaczenie
Customers	Dane klienta, email, hash hasła, typ klienta oraz data utworzenia.
Products	Wspólna tabela produktów bankowych. Kolumna Discriminator rozróżnia konto, kartę i kredyt.

Tabela	Znaczenie
Transactions	Historia operacji na kontach bankowych.
Transfers	Dane przelewów własnych i zewnętrznych.
OutboxMessages	Zdarzenia domenowe oczekujące na projekcję do bazy odczytu.

6.2 CalisiaReadDb

Tabela	Znaczenie
ProductTile	Kafelki produktów widoczne na dashboardzie: typ, numer, saldo, waluta i oznaczenie konta głównego.
TimelineEvent	Historia zdarzeń finansowych prezentowana w panelu użytkownika.
ProcessedOutboxMess	Rejestr przetworzonych wiadomości outbox, chroniący przed podwójną projekcją.

7 API REST



Rysunek 5: Mapa endpointów API

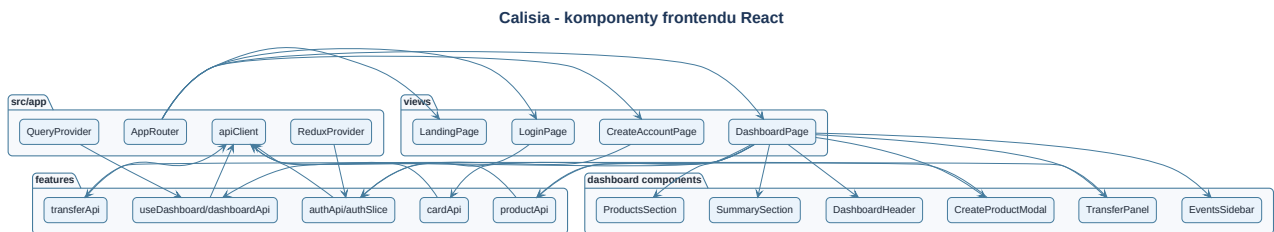
Diagram porządkuje endpointy REST według obszarów funkcjonalnych i wskazuje, które bazy danych są przez nie wykorzystywane.

Endpoint	Opis
POST /api/customers	Rejestracja klienta.
POST /api/customers/login	Logowanie i wydanie JWT.
GET /api/dashboard/{customerId}	Pobranie danych dashboardu z modelu odczytu.
POST /api/accounts	Otwarcie nowego konta bankowego.
POST /api/accounts/{id}/withdraw	Wypłata środków z konta.
POST /api/products	Dodanie produktu bankowego: konto, karta lub kredyt.
POST /api/products/{id}/early-repayment	Wcześniejsza spłata kredytu gotówkowego.
POST /api/transfers	Utworzenie przelewu własnego lub zewnętrznego.
POST /api/transfers/incoming	Symulacja wpływu przychodzącego.

Endpoint	Opis
POST /api/cards/{id}/payments/	Symulacja płatności kartą debetową lub kredytową.
POST /api/cards/{id}/repayment	Splata wykorzystanego limitu karty kredytowej.

8 Frontend

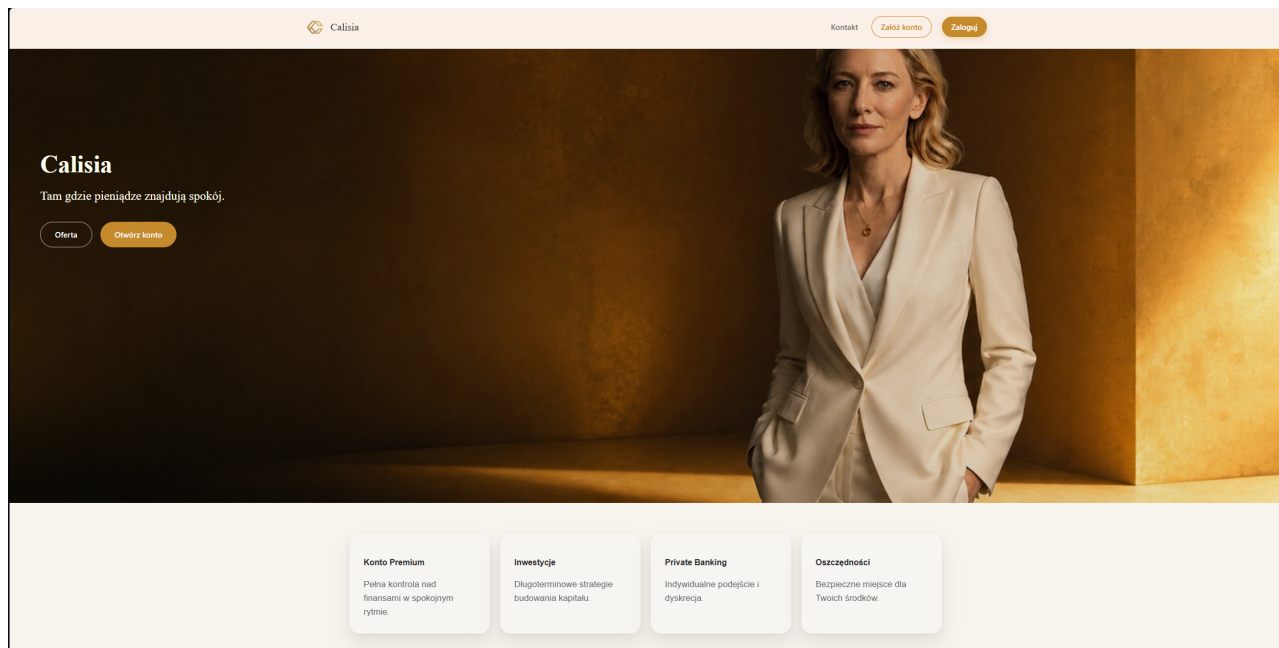
Frontend jest aplikacją React SPA. Routing realizuje **AppRouter**, stan sesji przechowywany jest w Redux Toolkit, a pobieranie i odświeżanie danych dashboardu obsługuje TanStack Query.



Rysunek 6: Komponenty frontendu

Diagram pokazuje zależności między routingiem, widokami, komponentami dashboardu oraz warstwą komunikacji z API.

8.1 Ekran aplikacji



Rysunek 7: Strona startowa aplikacji

Ekran wejściowy prezentuje markę Calisia i prowadzi użytkownika do logowania albo rejestracji nowego klienta.

Załącz konto w Calisia

Imię

Nazwisko

Adres e-mail

Hasło

Typ klienta

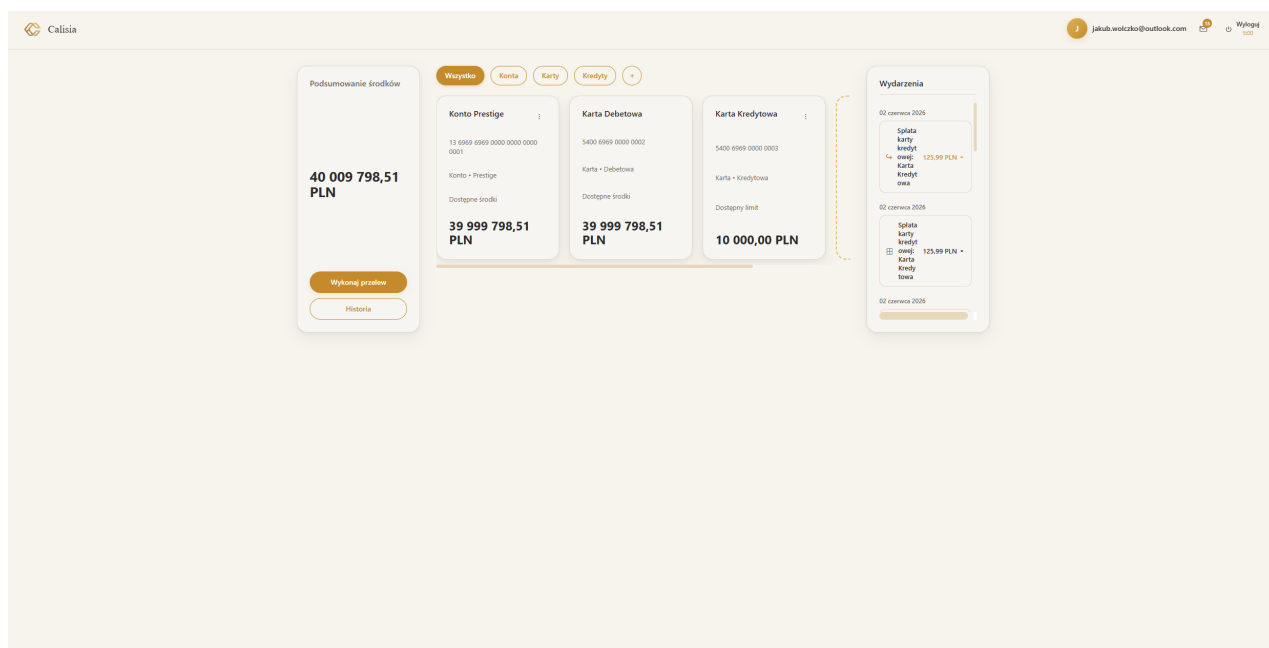
Standard

Wstecz

Załącz konto

Rysunek 8: Formularz rejestracji klienta

Formularz zbiera podstawowe dane klienta oraz typ klienta, na podstawie którego backend tworzy produkty startowe.



Rysunek 9: Dashboard klienta

Dashboard jest głównym widokiem roboczym klienta: prezentuje saldo, produkty, szybkie akcje i historię zdarzeń.

Karta • Debetowa

Do przelewów

Typ przelewu

Zewnętrzny

Konto źródłowe

Konto Prestige • 13 6969 6969 0000 0000 0001

Konto docelowe

Odbiorca

Tytuł przelewu

Kwota

Anuluj Wykonaj przelew

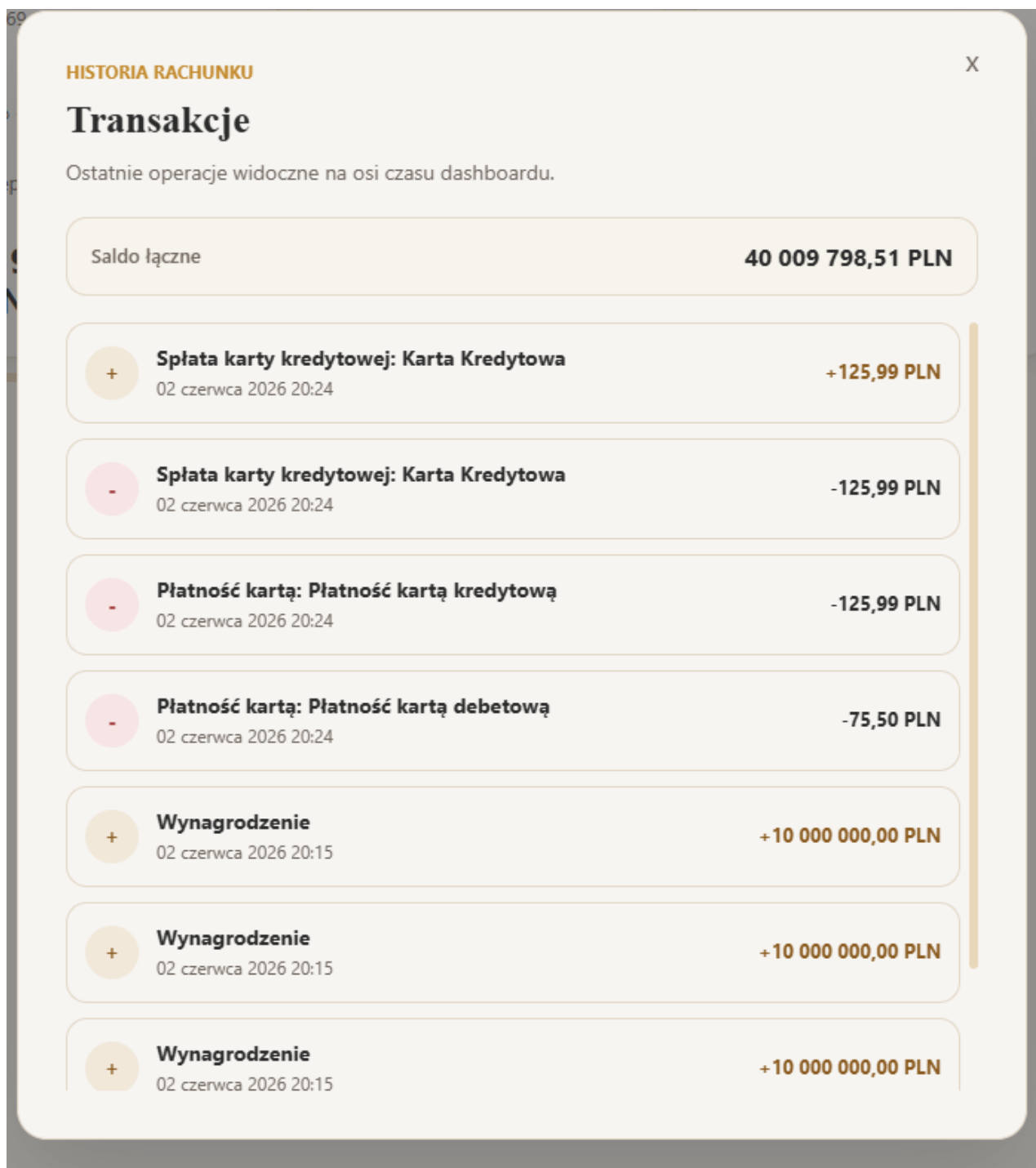
Rysunek 10: Panel tworzenia przelewu

Panel umożliwia wykonanie przelewu własnego lub zewnętrznego z walidacją konta źródłowego, odbiorcy, tytułu i kwoty.

The image shows a modal dialog titled "Otwórz nowy produkt" (Open new product) with a close button (X) in the top right corner. Below the title, there are three buttons: "Konto", "Karta", and "Kredyt". The "Kredyt" button is highlighted in orange. Below these buttons, the text "Wnioskowana kwota (PLN)" (Requested amount (PLN)) is displayed. Underneath is a text input field containing "0.00" and a currency selector showing "PLN" with up and down arrows. Below this, the text "Nazwa własna (opcjonalnie)" (Own name (optional)) is shown, followed by a text input field containing "np. Kredyt Gotówkowy" (e.g., Cash Loan). At the bottom of the modal is a large orange button labeled "Zatwierdź" (Confirm).

Rysunek 11: Dodanie produktu typu kredyt gotówkowy

Modal produktu pozwala wybrać kredyt gotówkowy i podać wnioskowaną kwotę, która po zatwierdzeniu zasila konto główne.

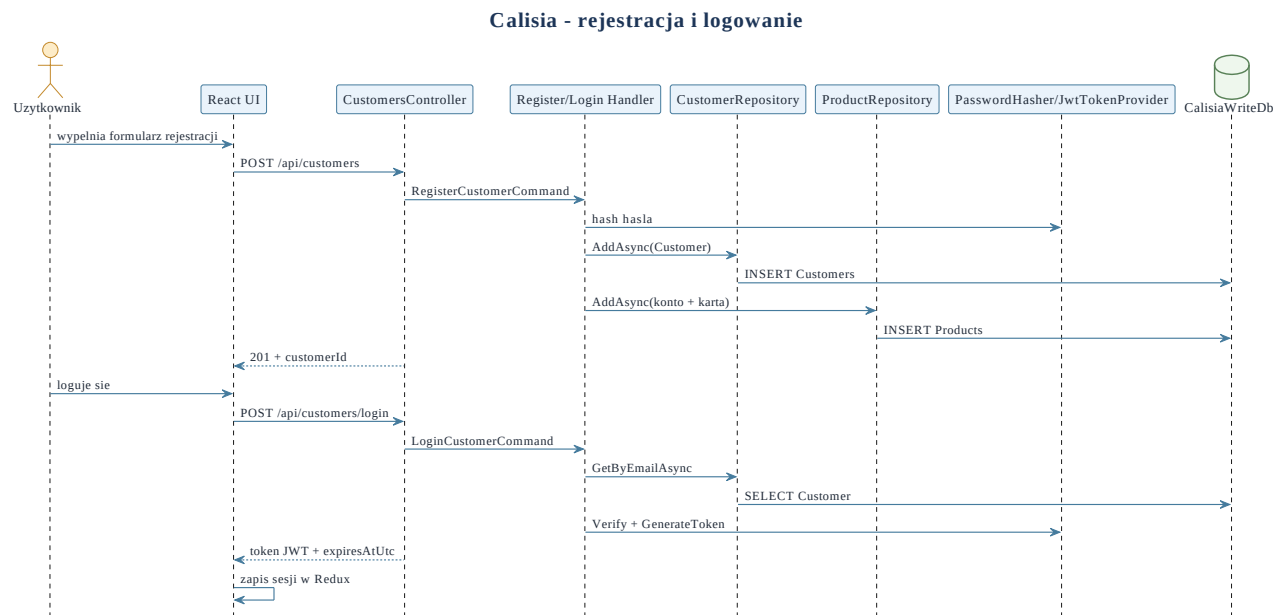


Rysunek 12: Historia transakcji i zdarzeń

Widok historii pokazuje zdarzenia finansowe generowane przez przelewy, wpłaty, wypłaty, płatności kartą i spłaty.

9 Przepływy procesów

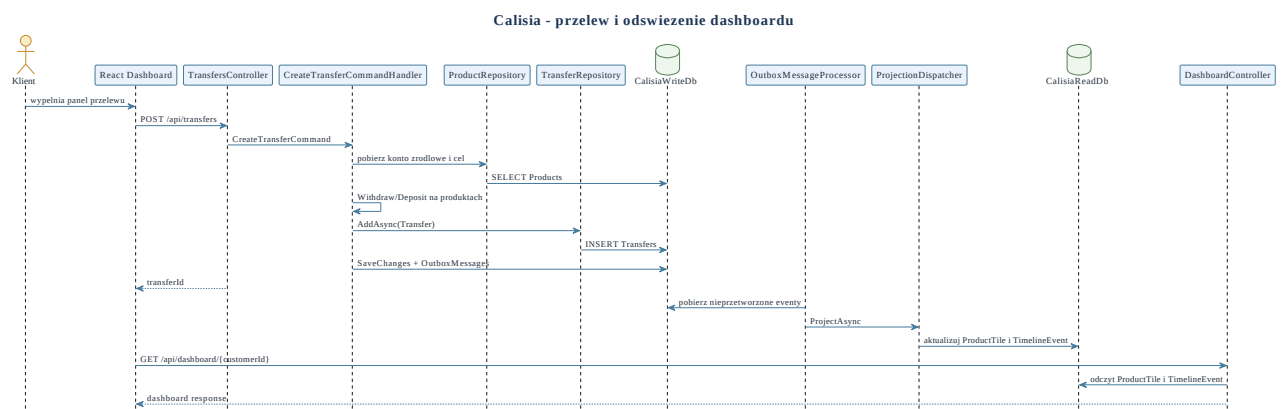
9.1 Rejestracja i logowanie



Rysunek 13: Sekwencja rejestracji i logowania

Diagram opisuje przejście od formularza rejestracji przez utworzenie klienta i produktów startowych do logowania JWT.

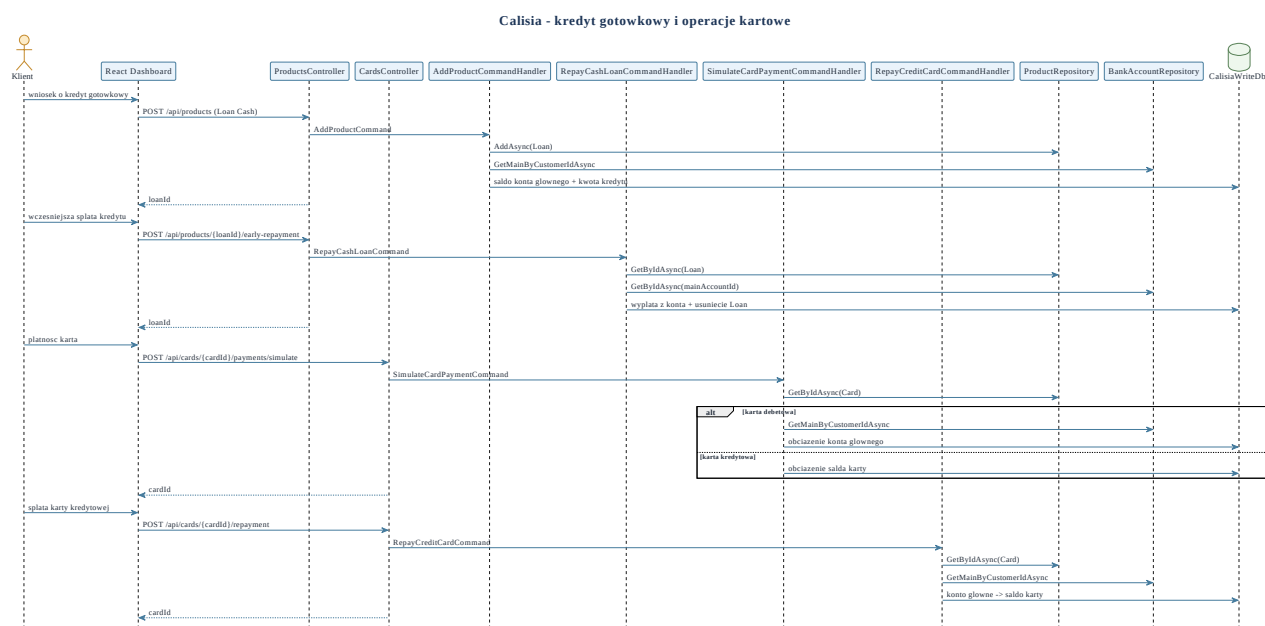
9.2 Przelew i dashboard



Rysunek 14: Sekwencja przelewu i odświeżenia dashboardu

Diagram pokazuje zapis przelewu w modelu write, publikację zdarzeń outbox i odświeżenie modelu read używanego przez dashboard.

9.3 Kredyt i operacje kartowe



Rysunek 15: Sekwencja kredytu gotówkowego i operacji kartowych

Diagram zestawia najważniejsze procesy finansowe: uruchomienie i spłatę kredytu, płatności kartą oraz spłatę karty kredytowej.

10 Mechanizm CQRS i Outbox

W systemie zastosowano rozdzielenie zapisu i odczytu. Komendy modyfikujące stan działają na bazie `CalisiaWriteDb`. Zdarzenia domenowe są zapisywane jako wiadomości w tabeli `OutboxMessages`. Procesor outbox przekazuje je do `ProjectionDispatcher`, który aktualizuje model odczytu.

Takie rozwiązanie pozwala:

- utrzymać bogaty model domenowy po stronie zapisu,
- uprościć odczyt danych dashboardu,
- ograniczyć ryzyko utraty zdarzeń dzięki zapisowi eventów w tej samej transakcji co zmiana domenowa,
- wykonywać projekcje idempotentnie dzięki tabeli `ProcessedOutboxMessage`.

11 Bezpieczeństwo

Autoryzacja użytkownika oparta jest o token JWT. Po zalogowaniu frontend zapisuje token i przekazuje go w nagłówku `Authorization: Bearer ....` Endpoint dashboardu dodatkowo sprawdza, czy identyfikator klienta w adresie odpowiada klientowi z tokenu.

Hasła nie są przechowywane jawnie. Backend używa hashera PBKDF2, a w bazie przechowywany jest wynik hashowania. Middleware obsługi wyjątków mapuje błędy domenowe i aplikacyjne na odpowiedzi HTTP.

12 Testy i narzędzia

Projekt zawiera testy jednostkowe oraz integracyjne backendu. Testy jednostkowe pokrywają między innymi:

- tworzenie produktów bankowych,
- kredyt gotówkowy i jego spłatę,
- płatności kartą debetową i kredytową,
- spłatę karty kredytowej,
- przelewy,
- hashowanie haseł i tokeny JWT,
- projekcje modelu odczytu.

Do ręcznego testowania endpointów przygotowano kolekcję Bruno. Zawiera ona żądania dla klientów, dashboardu, produktów, przelewów oraz kart.

13 Wnioski

Calisia Banking App jest aplikacją demonstracyjną pokazującą bankowość elektroniczną z wyraźnym podziałem warstw, modelem domenowym oraz odseparowanym modelem odczytu. Najważniejszą cechą architektury jest połączenie DDD, CQRS i outbox, które umożliwia spójne przetwarzanie zdarzeń finansowych oraz szybkie prezentowanie danych dashboardu.

Frontend zapewnia użytkownikowi prosty przepływ od rejestracji i logowania po codzienną obsługę rachunku, produktów, kart, kredytu oraz historii operacji. Backend koncentruje reguły biznesowe w domenie i udostępnia je przez czytelne endpointy REST.